For the Backend Developer

You are the central nervous system of the project, connecting the brain (blockchain) to the face (frontend).

- **Today (Sep 17):**
    1. **Install Environment:** Install **Node.js** and **VS Code**.
    2. **Setup Express Project:** Create a new project folder, run `npm init -y`, and then `npm install express`. Create a main file `server.js`.
    3. **Create Mock API:** In `server.js`, set up two API endpoints:
        - `POST /api/herbs`: For now, this should just log the request body to the console and return a success message like `{ "status": "success", "batchId": 1 }`.
        - `GET /api/herbs/:id`: This should return a hardcoded JSON object with fake herb batch data.
    4. **Handoff to Frontend Dev:** Start the server (`node server.js`) and share the API URLs (e.g., `http://localhost:3001/api/herbs`) with the Frontend Developer so they can start building against your mock API.
- **Tomorrow (Sep 18):**

    1. **Receive Handoff:** Get the **contract address** and **ABI file** from the Blockchain Dev.
    2. **Install Libraries:** Stop the server and run `npm install ethers`.
    3. **Connect to Blockchain:** In your `server.js`, add the code to connect to the local Hardhat blockchain using `ethers.js`. You'll need the contract address and ABI for this.
    4. **Implement Real Logic:**
        - Update the `POST /api/herbs` endpoint to call the `createHerbBatch()` function on the actual smart contract.
        - Update the `GET /api/herbs/:id` endpoint to call the `getHerbHistory()` function on the smart contract and return the real data.
- **Sep 19 - 20:**

    1. **Integrate & Debug:** Work side-by-side with the Frontend Developer. Use `console.log` extensively to check the data flowing from the frontend to your API and then to the smart contract. Fix any bugs that come up.
    2. **Prepare for Deployment:** Clean up your code and add comments. Ensure all necessary environment variables (like the contract address) are clearly defined.